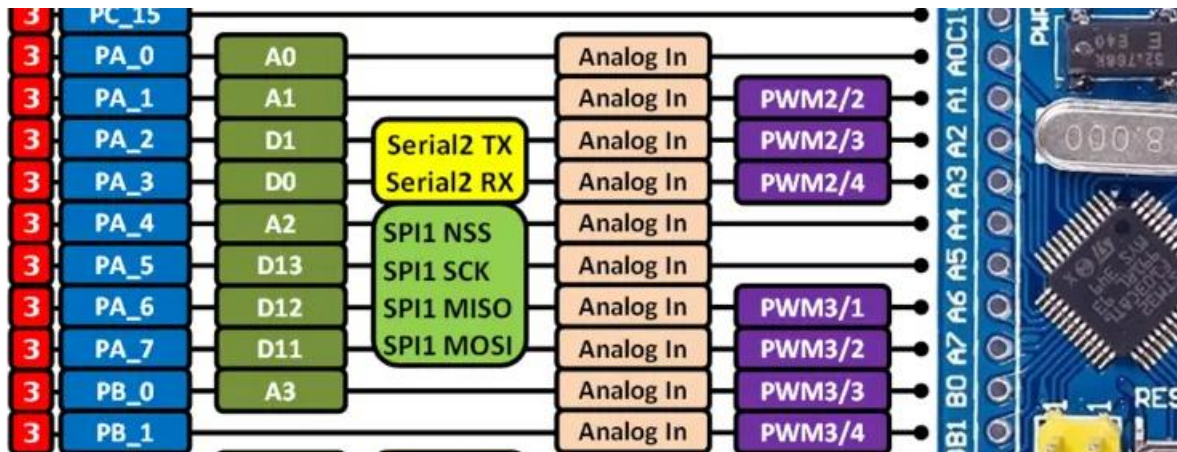


Conversão Analógico-Digital no STM32F103C8T6



Características do ADC

- 2 conversores ADC de 12 bits (ADC1 e ADC2)
- Até 16 canais externos
- Taxa de amostragem de até 1 MHz
- Tensão de referência: 0 a 3,3V (padrão)
- Modos de conversão: contínuo, único ou por injeção



Criando o Projeto

1. Abra STM32CubeIDE

2. Crie novo projeto com STM32F103C8T6

3. Configure clock e periféricos

4. Inicialize todos os periféricos no modo padrão

Configuração do ADC

- Ative ADC1 na aba "Analog"

- Parâmetros:

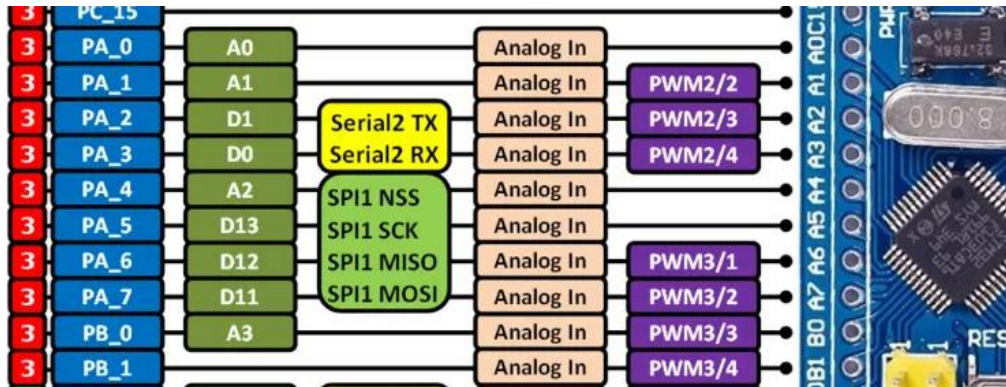
- Modo: Single/Continuous

- Resolução: 12 bits

- Alinhamento: à direita

- N° de conversões: 1
(caso aquisite apenas 1 ADC)

Seleção de Pinos



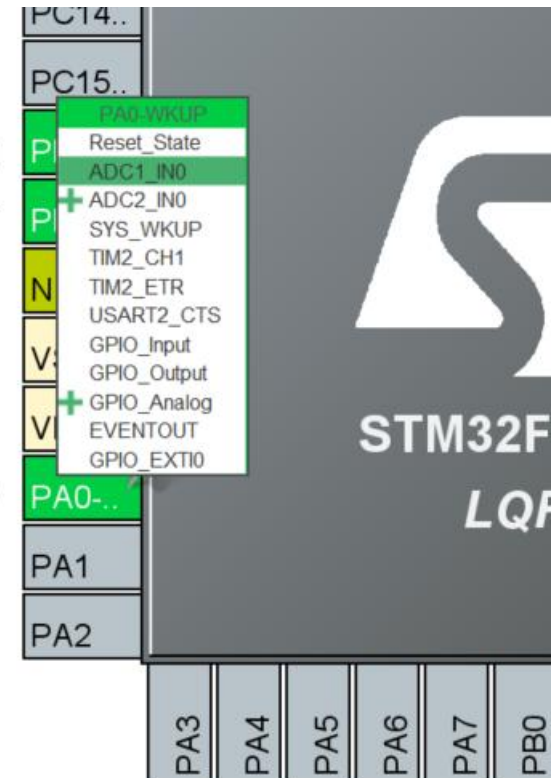
RCC_OSC_IN

RCC_OSC_OUT

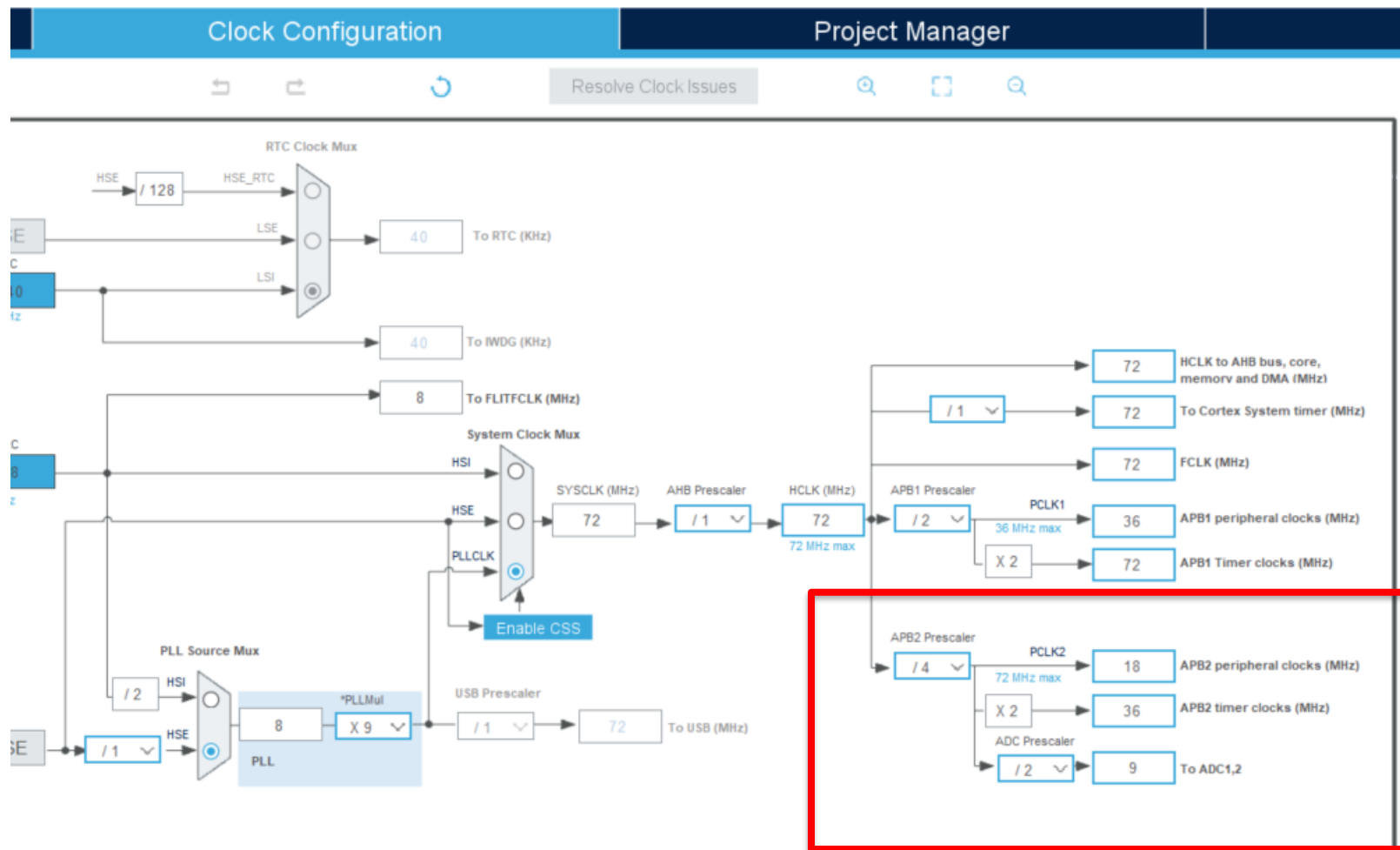
ADC1_IN0

Escolha o pino desejado entre as opções disponíveis

Escolha entre configurar como:
ADC1 ou ADC2

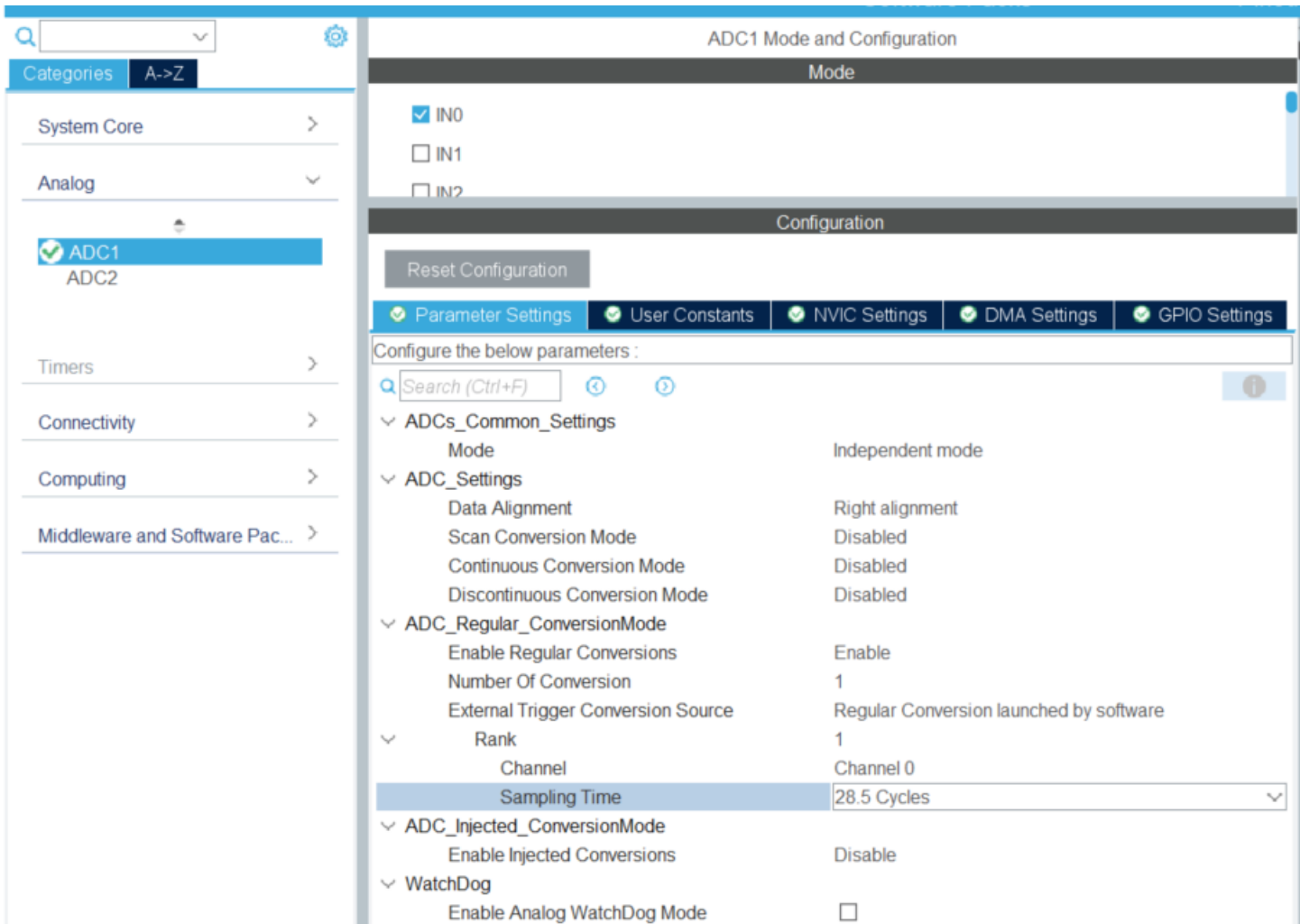


Configuração do Clock



Recomendação: ADC clock máximo de 14MHz

Leitura no Modo Polling



Configure o tempo de amostragem (geralmente 28.5 ciclos)

Leitura no Modo Polling

```
HAL_ADC_Start( &hadc1); // Inicia a conversão ADC  
HAL_ADC_PollForConversion(&hadc1, 10);  
// Espera a conversão ser concluída  
valorADC = HAL_ADC_GetValue(&hadc1);  
// Lê o valor convertido - Devolve um numero entre 0  
e 4096  
HAL_ADC_Stop(&hadc1); //Desabilita o periférico
```

```
// Variável para armazenar o valor convertido
uint16_t valorADC;

HAL_ADC_Stop(&hadc1);
HAL_ADCEx_Calibration_Start(&hadc1);

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_ADC_Start( &hadc1); // Inicia a conversão ADC
    HAL_ADC_PollForConversion(&hadc1, 10); // Espera a conversão ser concluída
    valorADC = HAL_ADC_GetValue(&hadc1); // Lê o valor convertido - Devolve um número
    HAL_ADC_Stop(&hadc1);
}
```

Leitura no Modo Continuous

The screenshot displays the STM32CubeMX software interface for configuring the ADC1. The left sidebar shows the project structure with 'ADC1' and 'ADC2' selected under the 'Analog' category. The main window shows the 'ADC1 Mode and Configuration' settings.

ADC1 Mode and Configuration

- Mode**
 - ☒ IN0
 - ☐ IN1
- Configuration**
 - Reset Configuration
 - Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

ADCs_Common_Settings

- Mode: Independent mode

ADC_Settings

- Data Alignment: Right alignment
- Scan Conversion Mode: Disabled
- Continuous Conversion Mode: Enabled
- Discontinuous Conversion Mode: Disabled

ADC_Regular_ConversionMode

- Enable Regular Conversions: Enable
- Number Of Conversion: 1
- External Trigger Conversion Source: Regular Conversion launched by software

ADC_Injected_ConversionMode

- Enable Injected Conversions: Disable

WatchDog

- Enable Analog WatchDog Mode: ☐

Pinout Diagram

- CC_OSC_IN: PD0-..
- CC_OSC_OUT: PD1-..
- NRST: NRST
- VSSA: VSSA
- VDDA: VDDA
- ADC1_IN0: PA0-..
- ADC2_IN1: PA1
- PA2
- PA3

Leitura no Modo Continuous

HAL_ADC_Start(&hadc1);

- Inicia o modulo ADC na “void main”.

adcValue = HAL_ADC_GetValue(&hadc1);

- No loop While, ou chamada de função desejada retorna o valor de conversão armazenado.

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_ADC1_Init();
HAL_ADC_Start(&hadc1);

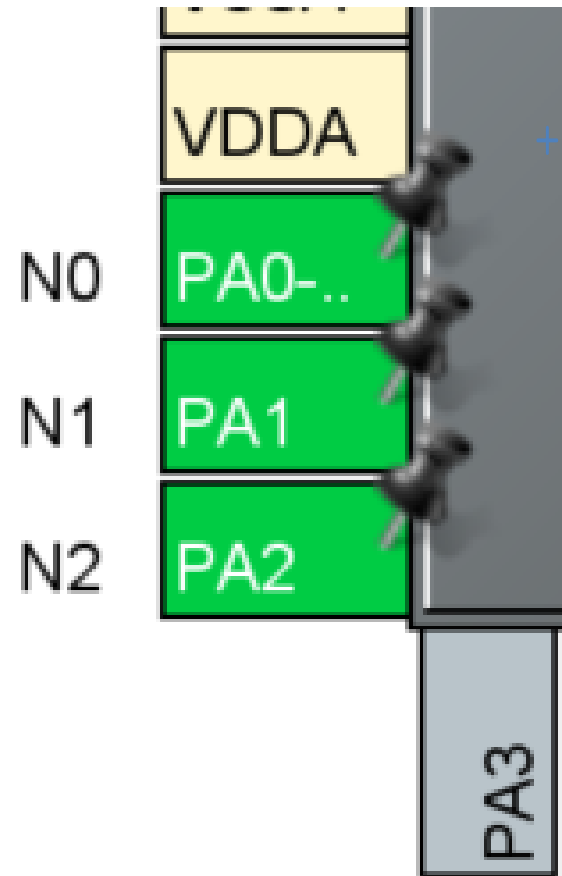
LCD16X2_Init(MyLCD);
LCD16X2_Clear(MyLCD);
LCD16X2_Set_Cursor(MyLCD, 1, 1);
LCD16X2_Write_String(MyLCD, "PROF Mozart");
HAL_Delay(2000);

while (1)
{
    adcValue = HAL_ADC_GetValue(&hadc1);
```

Leitura com DMA

Indicada para a leitura rápida de vários pinos, atribuindo uma taxa de amostragem padrão.

Ideal para a leitura de múltiplos sensores.



Leitura com DMA

The screenshot displays the STM32CubeMX configuration tool for the ADC1 peripheral. The left sidebar shows the project hierarchy with 'ADC1' selected under the 'Analog' category. The main window is titled 'ADC1 Mode and Configuration' and features several tabs: 'Parameter Settings', 'User Constants', 'NVIC Settings', 'DMA Settings' (which is active), and 'GPIO Settings'. A 'Reset Configuration' button is located above the tabs. Below the tabs, a table lists the configured DMA requests:

DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 1	Peripheral To Memory	Medium

At the bottom, the 'DMA Request Settings' section is visible. It includes an 'Add' button (highlighted with a red box) and a 'Delete' button. The 'Mode' dropdown menu is set to 'Circular' (also highlighted with a red box). Other settings include 'Increment Address' (unchecked), 'Data Width' (set to 'Word', highlighted with a red box), and 'Peripheral' (set to 'Word').

Leitura com DMA

The screenshot displays the STM32CubeMX configuration tool. The left sidebar shows the project hierarchy with 'ADC1' selected under the 'Analog' category. The main window is titled 'ADC1 Mode and Configuration' and shows the 'DMA Settings' tab. The 'Scan Conversion Mode' is set to 'Enabled'. The 'ADC_Regular_ConversionMode' section is expanded, showing three ranks with their respective channels and sampling times.

Mode	Configuration
☒ IN0 ☒ IN1 ☒ IN2	
Reset Configuration	
☒ Parameter Settings	☒ User Constants ☒ NVIC Settings ☒ DMA Settings ☒ GPIO Settings
Configure the below parameters :	
Mode	Independent mode
ADC_Settings	
Data Alignment	Right alignment
Scan Conversion Mode	Enabled
Continuous Conversion Mode	Disabled
Discontinuous Conversion Mode	Disabled
ADC_Regular_ConversionMode	
Enable Regular Conversions	Enable
Number Of Conversion	3
External Trigger Conversion Source	Regular Conversion launched by software
Rank	1
Channel	Channel 0
Sampling Time	239.5 Cycles
Rank	2
Channel	Channel 1
Sampling Time	239.5 Cycles
Rank	3
Channel	Channel 2
Sampling Time	239.5 Cycles
ADC_Injected_ConversionMode	
Enable Injected Conversions	Disable

Leitura com DMA

```
uint32_t adc_buffer[2];
```

Crie um vetor para armazenar as conversões.

```
HAL_ADC_Start_DMA(&hadc1, adc_buffer, 2);
```

Chame a função para inicializar a conversão

Conversão para Tensão

```
uint16_t adcValue; // Variável para armazenar o  
resultado da conversão analógica para digital  
// Varia de 0 a 4095 (12bits)  
float Volt; //Variável para armazenar o valor em Volt  
//Varia de 0 a 3.3 Volts  
Volt = adcValue *( 3.3 / 4095);
```

Boas práticas

- Calibre o ADC antes do uso:
`HAL_ADCEx_Calibration_Start(&hadc1);`
- Use capacitores de desacoplamento próximos aos pinos analógicos
- Mantenha sinais digitais longe dos analógicos
- Para sinais ruidosos, considere:
 - Média de múltiplas leituras
 - Filtros digitais simples

Referências

- STM32F103xx Reference Manual (RM0008)
- STM32F103 Datasheet
- STM32CubeF1 HAL Documentation
- STMicroelectronics Application Notes (AN3116, AN2834)